

8. Koordination

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

8.1 Vorschau

- Auswahlalgorithmen
- Gegenseitiger Ausschluss
- Konsistente Schnitte
- Verteilter Schnappschuss

8.1 Auswahlalgorithmen

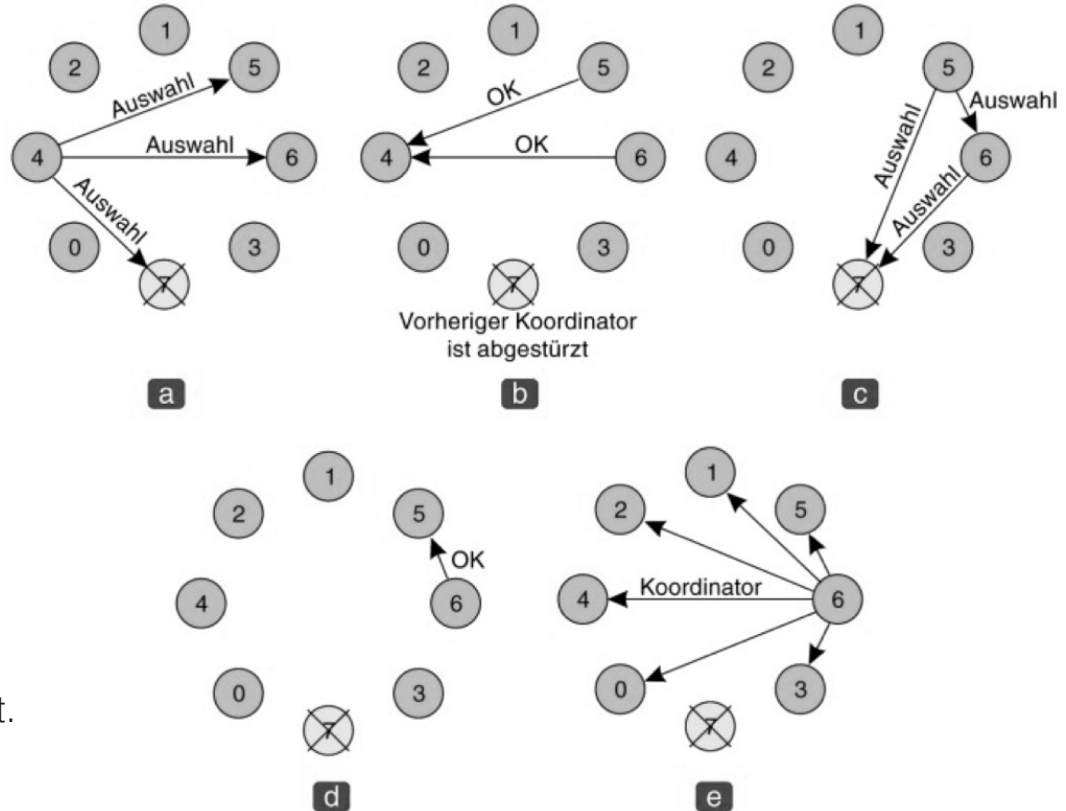
- Oft wird in verteilten Algorithmen ein Prozess benötigt, der eine spez. Rolle übernimmt, z.B. Koordinator, Initiator oder Monitor.
- Ziele eines Auswahlalgorithmus:
 - Einen solchen Prozess unter vielen gleichartigen zu bestimmen
 - Am Ende der Wahl sind sich alle einig, wer der neue Koordinator ist
- Grundprinzip:
 - Jeder Prozess hat eine Nummer, die allen anderen Prozessen bekannt ist
 - Kein Prozess weiß, welcher andere Prozess gerade läuft oder nicht
 - Alle Algorithmen wählen den Prozess mit der höchsten Nummer aus

Bully-Algorithmus (Garcia-Molina, 1982)

- Wenn ein Prozess P feststellt, dass der augenblickliche Koordinator nicht mehr reagiert, startet er den Auswahlprozess.
- P schickt **Auswahl-Nachricht an alle Prozesse mit höherer ID**
 - Bekommt P keine Antwort, ist er der neue Koordinator
 - Bekommt P eine Antwort, ist seine Aufgabe erledigt.
- Antwortende Prozesse übernehmen die Aufgabe und halten wieder eine Wahl ab (rekursiv).
- Der „Stärkste“ (engl. bully) bleibt übrig, und schickt am Ende an alle **eine Koordinator-Nachricht.**

Bully-Algorithmus (Garcia-Molina, 1982) - Beispiel

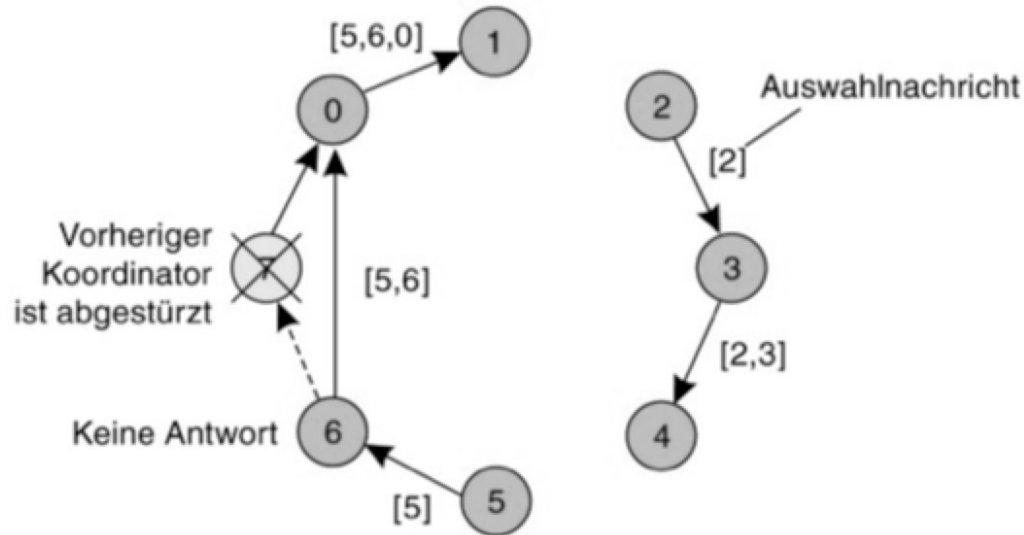
- a) P_4 startet Wahl
- b) P_5 und P_6 antworten mit OK $\rightarrow P_4$ stoppt
- c) P_5 und P_6 starten jeder eine Wahl ab
- d) P_6 antwortet P_5 mit OK $\rightarrow P_5$ stoppt
- e) P_6 bekommt keine Antwort, gewinnt somit und schickt allen die „Koordinator“-Nachricht.



- Prozesse sind durch einen logischen Ring verbunden, aufsteigenden nach Prozessnummer
- Wenn ein Prozess feststellt, dass der Koordinator nicht mehr läuft, sendet er eine **Auswahl-Nachricht** auf den Ring:
 - Nachricht speichert Liste aller Prozesse (Initiator der Wahl trägt sich als Erster ein).
 - Jeder weitere aktive Prozess fügt sich selbst in die Liste ein.
- Wenn Auswahl-Nachricht wieder beim Initiator eintrifft, wandelt er diese in eine **Koordinator-Nachricht** um:
 - Enthält neuen Koordinator und alle aktiven Prozesse
 - Wird dann nochmals auf dem Ringe durchgereicht

Ring-Algorithmus - Beispiel

- P_2 & P_5 senden gleichzeitig Auswahl-Nachricht
- Beide Nachrichten kreisen im Ring
- Alle lernen, dass der neue Koordinator P_6 ist

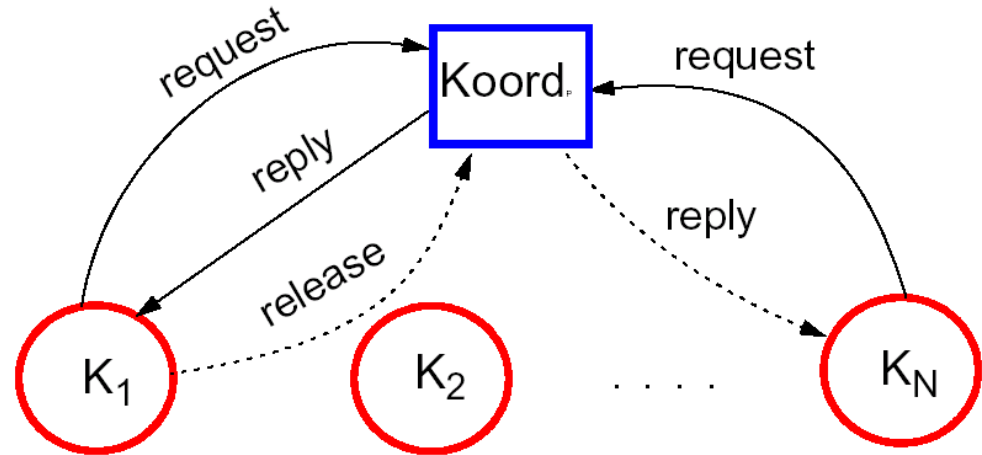


8.2 Gegenseitiger Ausschluss

- Nur ein Prozess darf zu einem Zeitpunkt im kritischen Abschnitt sein
 - z.B. zur Koordinierung gemeinsam genutzter Betriebsmittel
- Verhungern muss vermieden werden
 - Jeder Prozess sollte irgendwann zum Zug kommen
- Fairness optional beachten
 - z.B. kein Prozess darf bevorzugt werden

Zentrale Lösung

- Zentraler Koordinator regelt Zugang zu kritischem Abschnitt
 - Betreten des kritischen Abschnitts
 - Sende `request`-Nachricht an Koordinator
 - Warte auf `reply`-Nachricht
 - Verlassen des kritischen Abschnitts:
 - Sende `release`-Nachricht an den Koordinator.



Zentrale Lösung - Bewertung

- Zentraler Koordinator = Flaschenhals & zentraler Fehlerpunkt (aber einfach implementierbar)
- Zwei Nachrichten für den Eintritt in den kritischen Abschnitt (aus Sicht eines anfragenden Knotens)
- Fairness: Koordinator kann einfach eine faire Zuteilungsstrategie implementieren

Verteilte Lösung nach Lamport (1978)

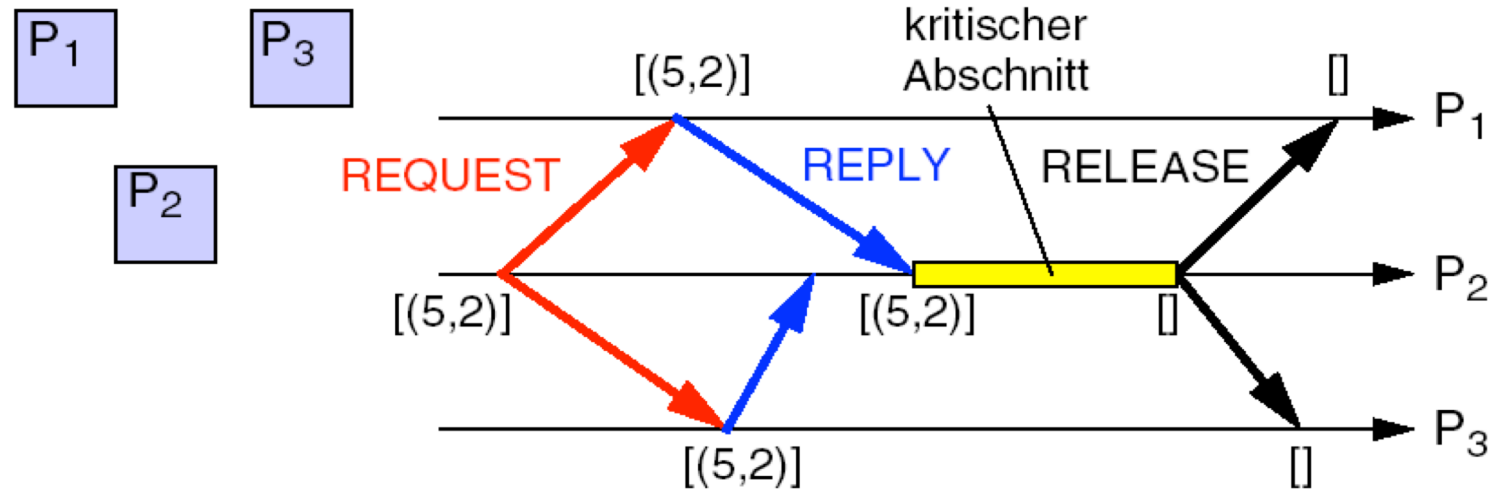
- Annahmen:
 - Jeder Prozess besitzt logische Lamport-Uhr
 - Zuverlässige Fifo-Kanäle zwischen allen Prozessen.
- Grundprinzip:
 - Jeder Prozess verwaltet eine nach Zeitstempeln geordnete Nachrichtenwarteschlange
 - Anforderungen und Freigaben werden per Multicast an alle Teilnehmer gesendet
 - Erst wenn alle Prozesse ihr OK gegeben haben, kann ein Prozess die kritische Region betreten.

Verteilte Lösung nach Lamport - Implementierung

- Anforderung senden: **REQUEST** mit Zeitstempel in eigene Schlange einreihen und an alle senden
- Anford. empfangen: **REQUEST** in eigene Schlange einreihen und **REPLY** (Anfragebestätigung) an anfragenden Prozess senden
- Freigabe senden: eigenen **REQUEST** aus eigener Schlange entfernen und **RELEASE** (Freigabe) an alle senden
- Freigabe empf.: **REQUEST** aus eigener Schlange löschen
- Bedingungen für Betreten des kritischen Abschnitts:
REPLY-Nachricht von allen anderen Knoten empfangen und eigener **REQUEST** am Kopf der Warteschlange

Verteilte Lösung nach Lamport - Beispiel 1

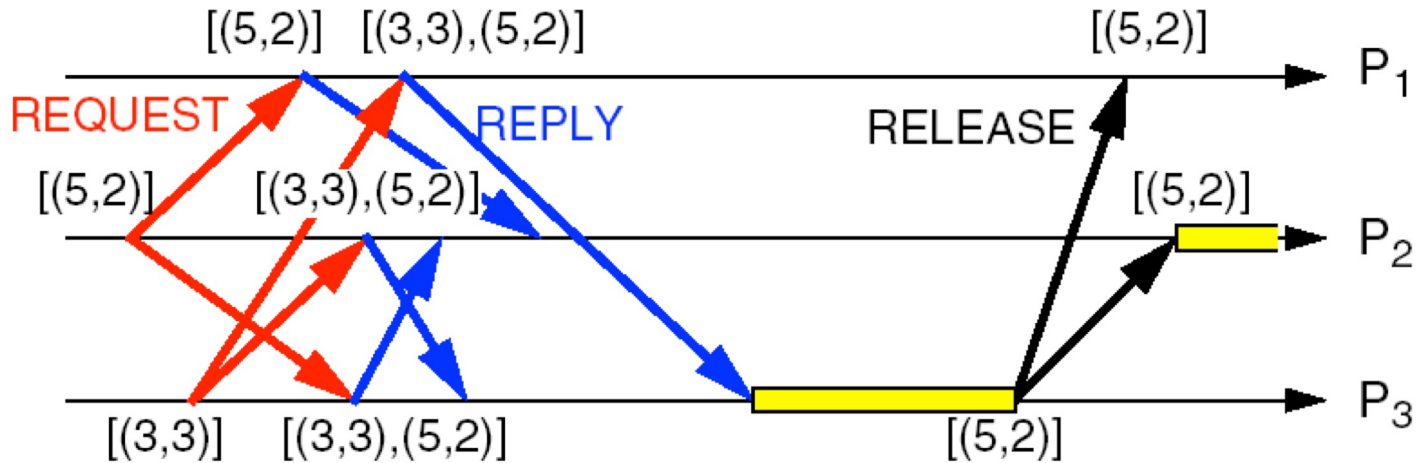
- Ein **REQUEST** (Lamport-Zeit, PID) von P_2



Verteilte Lösung nach Lamport - Beispiel 2

- Zwei parallele REQUESTs

- Gleiche Sicht durch gleich sortierte Warteschlangen
- FiFo-Kanäle garantieren, dass alle vor REPLY gesendeten REQUESTS eines Prozesses vorher angekommen (wichtig).



Verteilte Lösung nach Lamport - Bewertung

- $3(N-1)$ Nachrichten für den Eintritt in den kritischen Abschnitt
 - Versenden der Anforderung an $(N - 1)$ Knoten
 - $(N - 1)$ Knoten versenden ein Reply
 - $(N - 1)$ Knoten versenden ein Release
- Koordinierungsverzögerung: eine Nachrichtenlaufzeit für RELEASE
- Gefahr des Verhungerns, falls ein Knoten ausfällt.

Verbesserte Lösung nach Ricart und Agrawala

- G. Ricart, A. K. Agrawala, „An Optimal Algorithm for Mutual Exclusion in Computer Networks“, Communications of the ACM, Vol. 24, No. 1, 1981.
- Bedingung: Zuverlässiger Nachrichtenkanal, jedoch keine FIFO-Garantie notwendig.
- Ziel: weniger Nachrichten als bei dem Ansatz von Lamport.
 - Wird erreicht durch Kombination von REPLY & RELEASE.

- Senden eines Eintritt-Wunsches: REQUEST an alle schicken.
- Empfangen eines Eintritt-Wunsches
 - Antworten mit einer **OK-Nachricht**, falls der Empfänger:
 - Weder selbst den kritischen Abschnitt betreten will,
 - noch ihn gerade betreten hat,
 - oder selbst eine Anforderung gestellt hat, aber der Zeitstempel der empfangenen Anforderung kleiner ist, als die der eigenen Anforderung.
 - Ansonsten wird die Antwort mit der OK-Nachricht aufgeschoben

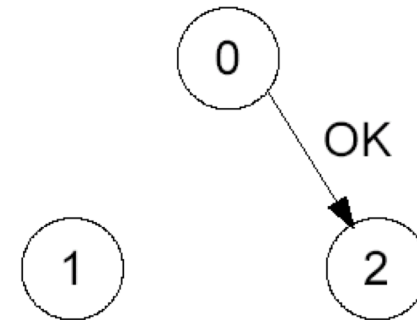
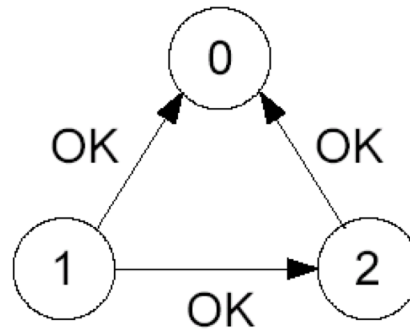
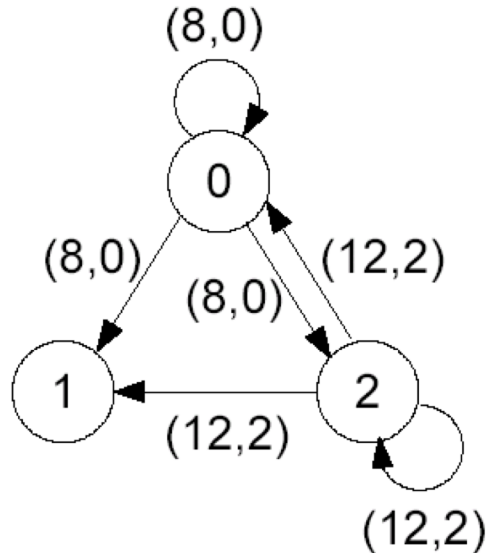
- **Betreten des kritischen Abschnitts:**
 - Wenn eigener REQUEST von allen anderen mit einer OK-Nachricht akzeptiert wurde
- **Verlassen des kritischen Abschnitts:**
 - Alle aufgeschobenen OK-Nachricht verschicken.

Beispiel: Requests (Lamportzeit, PID)

0 und 2 wollen in den kritischen Abschnitt

0 betritt den kritischen Abschnitt

0 verlässt und 2 betritt den kritischen Abschnitt



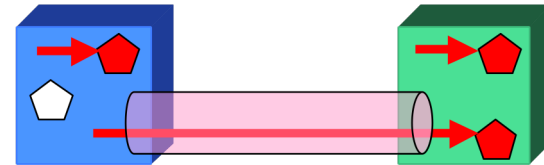
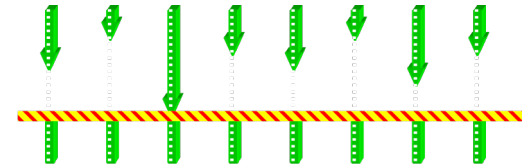
Bewertung

- Benötigt nur 2 ($N - 1$) Nachrichten pro Zugriff
 - Jeweils ($N - 1$) Nachrichten für REQUEST und OK
- Braucht keine FIFO-Kanäle
- Koordinierungsverzögerung: eine OK-Nachricht
- Verklemmungsgefahr, falls ein Knoten ausfällt

8.3 Globale Zustände

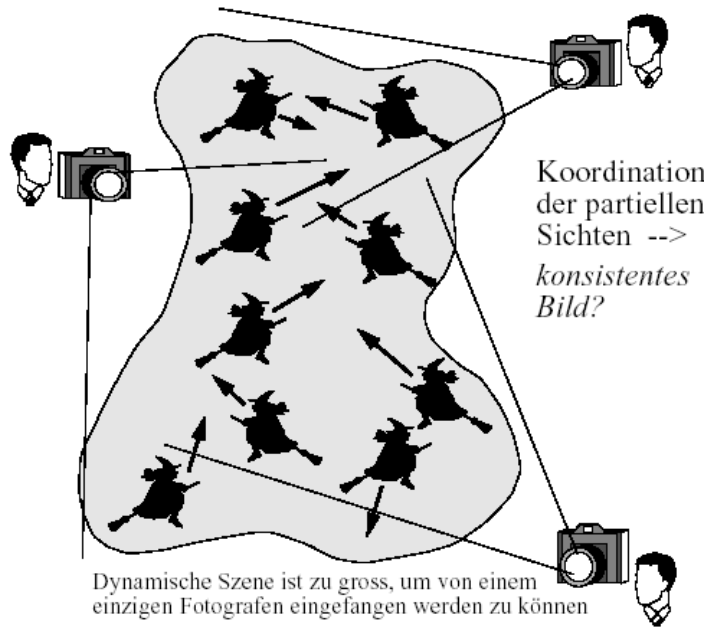
Motivation

- Terminierung einer Berechnung
 - Haben alle Knoten den (End-) Zustand erreicht (Barriere)?
- Verteilte Freispeichersammlung
 - Gibt es auf irgendeinem Knoten eine Referenz auf ein lokales Objekt?
- Verklemmungszustand erkennen
 - Gibt es einen zyklischen Wartezustand zwischen Knoten?
- Zum globalen Zustand gehört auch der Übertragungskanal



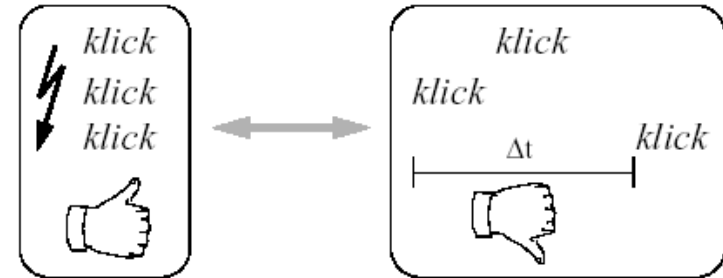
Synchronisierte globale Schnappschüsse?

- Herausforderung: „Momentaner“ Schnappschuss des globalen Zustandes, ohne das System anzuhalten.



Dynamische Szene ist zu gross, um von einem einzigen Fotografen eingefangen werden zu können

Bild: Prof. Mattern (ETH Zürich)

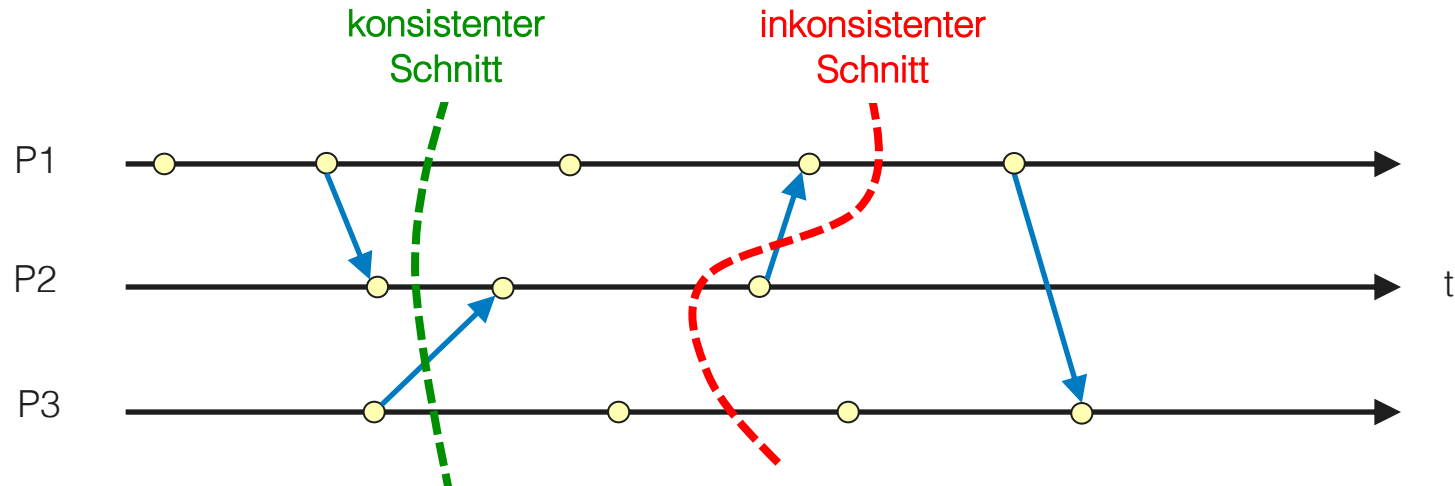


8.3.1 Konsistente Schnitte

Konsistente und inkonsistente Schnitte

- Problem: Schnappschuss zu einem bestimmten Zeitpunkt ist nicht realisierbar, da eine exakte Uhrensynchronisierung nicht möglich ist.
- Lösung: Schnappschuss sollte dennoch konsistent sein
→ Lösung: konsistente Schnitte mithilfe logischer Zeit
- Konsistenz bedeutet:
 - Für ein Ereignis e_x müssen alle vorhergehende Ereignisse e_i , von denen e_x kausal abhängt (happened before) einbezogen werden
 - Nachrichten „aus der Zukunft“ sind nicht möglich
→ Ursache immer vor der Wirkung.
 - konsistente Schnitte $\leftrightarrow$ kausaltreues Beobachten

Konsistente und inkonsistente Schnitte



8.3.2 Schnappschussverfahren von Chandy & Lamport

- Algorithmus-Eigenschaften:
 - Berücksichtigt auch die Kanalzustände
 - Arbeitet nebenläufig zu Anwendungen (hält diese also nicht an)
 - Jeder Prozess kann Initiator sein
- M. Chandy & L. Lamport, “Distributed snapshots: Determining global states of distributed systems“, in: ACM Transactions on Computing Systems, 3(1):63--75, 1984.

Voraussetzungen

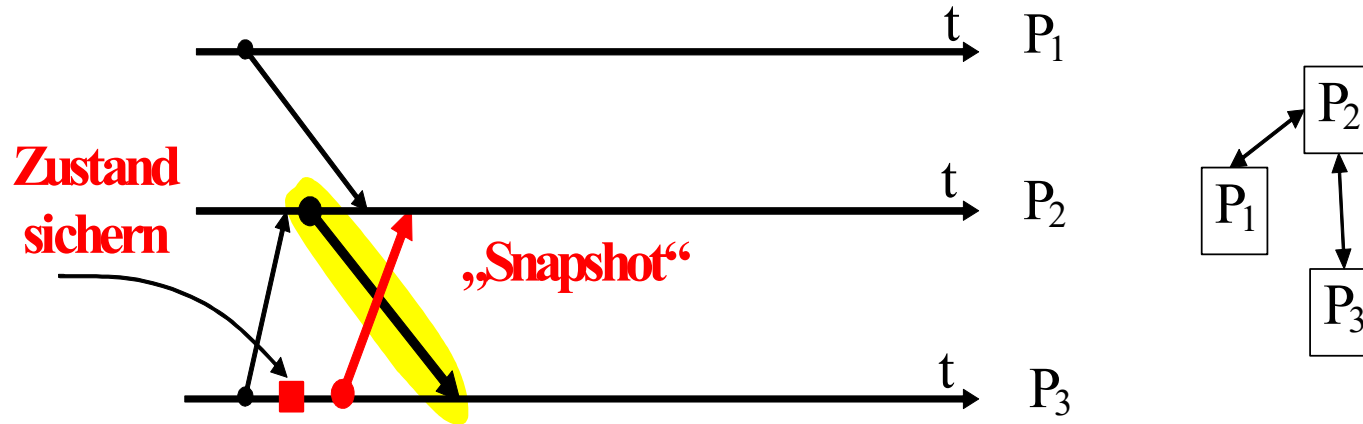
- Prozesse verbunden über bidirektionale Fifo-Kanäle (Paket-Übertragung und -Sequenz garantiert) → TCP
- Knoten & Nachrichten kennen zwei Zustände:
 - schwarz: Zustand vor Schnappschuss
 - rot: Zustand nach Schnappschuss
- Spezielle „Schnappschuss“-Nachricht oder eine rote Nachricht initiieren einen Schnappschuss.

Grundprinzip

- Rote Nachrichten „schieben“ schwarze Messages, die sich im Transit befinden, aus den Kanälen hinaus
→ letztere werden aufgezeichnet
→ sie gehören zum Schnappschuss
- Nach Empfang einer roten Nachricht, weiß ein Knoten, dass über diesen Kanal keine weiteren schwarzen Nachrichten mehr eingehen.

Empfang einer schwarzen Nachricht

- Falls Knoten rot gefärbt, Nachricht sichern (gehört zum Schnappschuss)
- Im Beispiel oben sichert P_3 schwarze Nachricht von P_2 , sonst „normal“ weiterarbeiten.

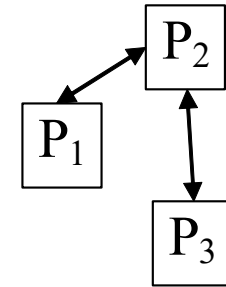
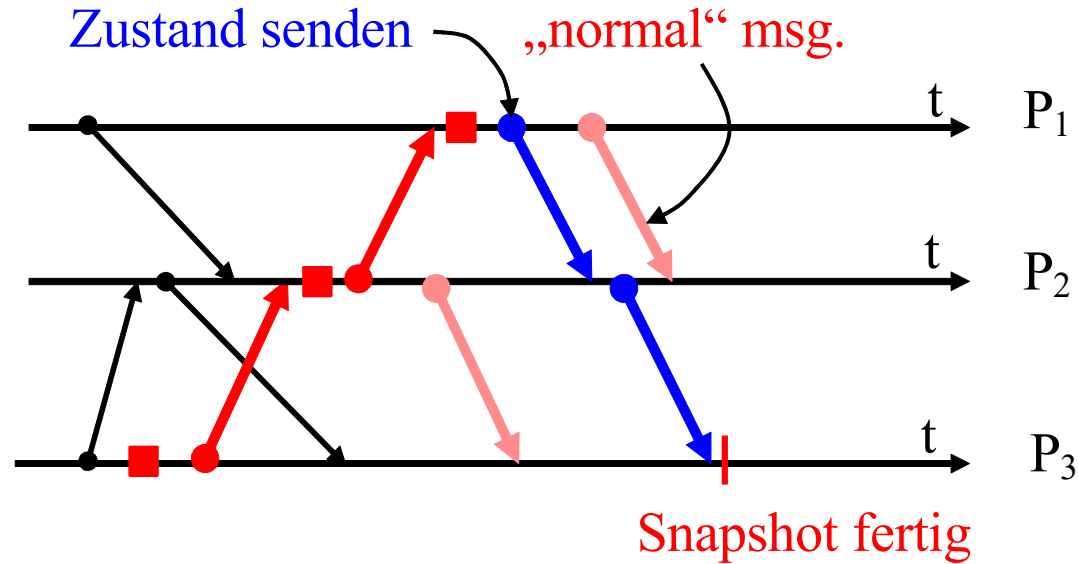


Empfang einer Schnappschuss- / roten Nachricht

- Falls erster Empfang
 - Lokalen Zustand sichern & Knoten rot färben
 - Schnappschussnachricht über alle Verbindungen weiterleiten (außer an den Knoten, von dem Nachricht empfangen)
- Jeder Knoten besitzt Flags für jede Verbindung (gesetzt, falls eine rote Nachricht eingeht).
- **Wenn alle Flags gesetzt, schickt Knoten Zustand** (+ zugehörige gesicherte schwarze Nachrichten) zum Initiator.
- Falls Knoten rot „gefärbt“, so werden **alle normale Nachrichten rot markiert**.

Empfang einer Schnappschuss- / roten Nachricht

- Beispiel



Bemerkungen

- Inkonsistenter Snapshot unmöglich, da schwarze Nachrichten, die zum Zustand gehören erkannt und mit gesichert werden.
- Außerdem kann nach einer roten Nachricht keine schwarze Nachricht mehr kommen → Fifo-Kanäle!

8.4 Zusammenfassung

Zusammenfassung

- Auswahlalgorithmen bestimmen eindeutig einen Koordinator
- Gegenseitiger Ausschluss kann zentral oder verteilt realisiert werden
 - zentrale Lösungen sind einfach
 - verteilte Verfahren vermeiden jedoch den Single Point of Failure
- Algorithmen die globale Zustände erfassen, müssen immer auch Nachrichten im Übertragungskanal berücksichtigen.
 - Chandy-Lamport-Schnappschuss erfolgt nebenläufig und gewährleistet kausaltreue Snapshots